

Solution 1: Map the Seconds Rule into "Grid2Op Time Steps" (Recommended)

Instead of forcing the simulator to run at a 3-second resolution (which it cannot do), you must mathematically translate your 3-second rule into Grid2Op's 5-minute framework.

Grid2Op actually uses this exact compromise for line overloads. In real life, a thermal overload might be tolerated for a few minutes. In Grid2Op, this is mapped to a parameter called `NB_TIMESTEP_OVERFLOW_ALLOWED = 2` (allowing it for 2 steps, or 10 minutes) before forcing a line disconnection.

You can apply the same logic to your rule validation engine:

The Real Rule: "Voltage spike > 1.1 p.u. can be tolerated for 3 seconds."

The Translated Grid2Op Check: If a voltage spike appears in an observation, you flag it, but you do not penalize the GNN unless that spike persists across a specific number of consecutive steps (or you use it to evaluate whether the GNN acts to clear it in the next step).

Solution 2: Change the Global Time-Delta Parameter

If you have a custom dataset or are building your own environment profiles (chronics), you can actually tell Grid2Op that each step represents a much smaller window of time.

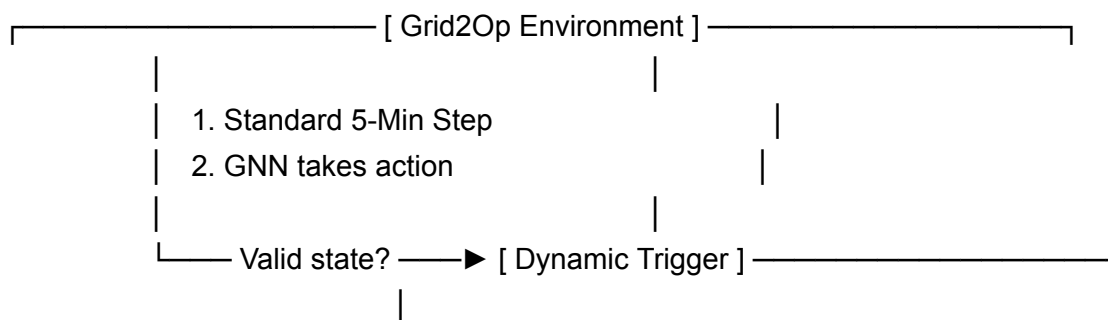
When loading or creating an environment, Grid2Op references a parameter called `delta_time_minutes` (stored in the environment's parameters/JSON configurations).

You can configure your environment parameters so that a step represents a shorter window, though dropping down to 3 seconds is heavily discouraged.

Why this is problematic: Grid2Op uses **Steady-State Power Flow** algorithms (like Newton-Raphson via LightSim2Grid or Pandapower). It calculates the grid equilibrium *after* a change happens. A 3-second voltage spike is a **transient dynamic phenomenon** (governed by differential equations, generator rotors, and sub-second wave propagation). Steady-state solvers completely ignore transient states.

Solution 3: "Two-Phase" Hybrid Simulation

If you want to keep your trained GNN but still test it against those sub-second, 3-second voltage manuals, you can build a hybrid validation architecture. Instead of replacing Grid2Op, you use it as the primary brain, and only pass critical states to a dynamic tool.



(If voltage bounds crossed)



[Mini-Dynamic Validation]

- Spin up a 5-second ANDES window
- Apply the rule: "Did it settle in 3s?"
- Return Pass/Fail to logs

How to Implement This Without Re-training: Run your simulation loop in Grid2Op: Let your GNN read the observation and take actions at the standard 5-minute interval. Intercept

"Near-Violation" Steps: If Grid2Op reports a voltage that is pushing the upper limits (e.g., 1.08

p.u.), flag this time step. Run a Shadow Dynamic Check: Take the network topology and the exact load/generation values from that Grid2Op step. Feed them into a quick, isolated script using a Python dynamic library (like ANDES or pandapower's short-circuit module). Introduce a fault (e.g., trip a line), simulate it for 5 seconds, and check if the voltage spike settles within 3

seconds. Log the Result: Do not feed this sub-second data back to the GNN. Just use it to verify:

"In this scenario, the GNN's grid state would have successfully tolerated a real-world transient."

Expanding on solution 3:

Rule Category	Why the Simulator Rejects It	How to Make It Compatible
Time-Bound & Fast Physics (Most Common)	It only sees 5-minute snapshots and misses split-second actions.	Wake up a fast simulator (like ANDES) just for these moments to check the fast math.
Frequency Limits	It assumes the grid frequency is always a perfect 50 Hz or 60 Hz.	Pass the grid data to the fast simulator to test how the grid reacts to frequency drops.
Relay & Hardware Settings	It looks at the overall grid power flow, not the internal switches of a single relay.	Translate the hardware rule into a grid limit (like making sure a power line never exceeds a generator's limit).
Strict Voltage Bounds	Test grids normally run a bit high. Strict rules cause alarms to ring constantly even when safe.	Change the rule to watch for sudden jumps from the normal voltage, instead of a strict hard limit.
Paperwork & Admin	It cannot read compliance reports, 48-hour notices, or 5-year studies.	These cannot be saved for the live shield. They must be permanently thrown out.

The Problem

Grid2Op only takes slow snapshots of the power grid. It misses fast physical changes, which means it cannot provide the data needed for most of the safety rules. Because of this **missing data, those rules had to be thrown out.**

The Solution

We use ANDES as an on-demand, high-speed camera. We only wake it up when the AI makes a decision. ANDES simulates the next few seconds in high detail, finally providing all the missing variables (like exact frequency and split-second timing).

The Safety Check

Now that ANDES has provided the proper data, those discarded rules become useful again. The system simply takes the highly detailed ANDES data and matches it against the rules. If the data breaks any of the rules, the system catches the danger and blocks the move.

Why This Changes Everything

If you only use the slow simulator for the final test phase, you get the best of both worlds:

- **Fast Learning:** The AI still learns using the quick 5-minute snapshot simulator.
- **Deep Testing:** The final exam uses the deep physics simulator to prove the AI is truly safe.
- **More Rules:** You can unlock all those split-second rules (like the 140-millisecond limits and frequency drops) because the final test can actually see them happen.

How to Build the Evaluation Bridge

With Claude Code helping you write the scripts, you can build this final testing phase much faster. Here is a simple path to set it up:

- **Run the Fast Simulator:** Let Grid2Op run the grid and have your trained AI make its choices.
- **Pause and Save:** When the AI makes a choice, pause the system and save the exact state of the grid.
- **Hand Off the Data:** Send that saved snapshot over to ANDES.
- **Run the Physics:** Have ANDES simulate just a few seconds of the grid's reaction to see if the AI's choice caused any fast crashes.
- **Check the Shield:** Run your large batch of new, fast-acting rules against the ANDES results to see if the AI passed or failed.

Solving the Missing Data Problem

The only real hurdle left is that ANDES needs detailed machine information to run. Since you are just testing a concept, you do not need perfect real-world numbers.

- Find a standard research grid that already has dynamic data built-in.
- Use generic, average numbers for generator weights and speeds.

How ANDES works:

Here is exactly how running ANDES connects to evaluating your new, fast-acting rules.

The missing link between the simulator and the rules is **time-series data**. Grid2Op only gives you a single, static picture. Your new rules (like the PRC-024 and PRC-029 rules you just shared) require continuous monitoring over milliseconds. ANDES creates that continuous monitor.

1.ANDES Generates the High-Resolution Trajectory:The Simulation.

When ANDES simulates the 3 seconds following a line break, it calculates the grid's physics millisecond by millisecond. Instead of outputting one static snapshot, it outputs a dense time-series matrix. This matrix contains the exact variables Grid2Op lacks, such as frequency_hz, rocof_hz_per_s, and dynamic voltage_pu for every bus at every millisecond.

Verification: You can verify this step is successful when ANDES outputs a time-series array showing the transient curves for voltage and frequency over the specified 3-second window.

2.The Shield Parses the Time-Series Data:Data Translation.

Your evaluation Python script (the Shield) takes that raw time-series output from ANDES and maps it to the variables your rules require. For example, if a rule needs to know time_seconds and voltage_pu, the Shield reads the ANDES matrix to find the exact depth of a voltage dip and measures exactly how many milliseconds it lasted.

Verification: You can verify this step by checking if your Shield script successfully extracts peak, minimum, and duration values from the ANDES arrays into standard Python variables.

3.The Shield Evaluates the Logic:Rule Checking.

The Shield loops through your massive list of fast-acting rules and checks their logic against the extracted data. For example, rule R_2273 dictates a disconnect if voltage_pu < 0.45 and time_seconds < 0.15. The Shield looks at the ANDES data: did the voltage at the bus drop below 0.45? Yes. Did it stay there for less than 0.15 seconds? No, it stayed for 0.20 seconds. The Shield then logs the rule violation.

Verification: You verify this step by confirming the Shield logs a clear "BLOCK" or "ALERT" when the ANDES data crosses the specific thresholds defined in the .jsonl rule file.

In short, ANDES does not evaluate the rules itself. ANDES just acts as a high-speed camera generating the dynamic data that your new rules demand. Your Python script (the Shield) acts as the referee, comparing the "video playback" from ANDES against the JSON rule logic to see if the AI's action caused a physical violation.